

Architecture Document

Private Network Service Platform · Computer Networks Course Project · Phase 1
Team X: Agrima (Laptop 1) and Gargi (Laptop 2) · 5 October 2026

We built a small private website service on two MacBooks. When you type <https://app.teamX.test> on Laptop 2, our own DNS server finds the address, an nginx server checks the connection is secure, and one of two backend servers sends back the answer. Nothing runs in the cloud; everything happens on our Wi-Fi network.

The big picture, in one line: Laptop 1 is the “front office” (DNS, security, traffic control and one backend), and Laptop 2 is the “customer” plus a second backend.

Team: Agrima (Laptop 1) and Gargi (Laptop 2). The project brief allows teams of two to combine roles, so our two laptops cover all four machine roles.

1. Network topology

Both laptops share one Wi-Fi LAN; Laptop 1 hosts DNS, the edge and Backend A

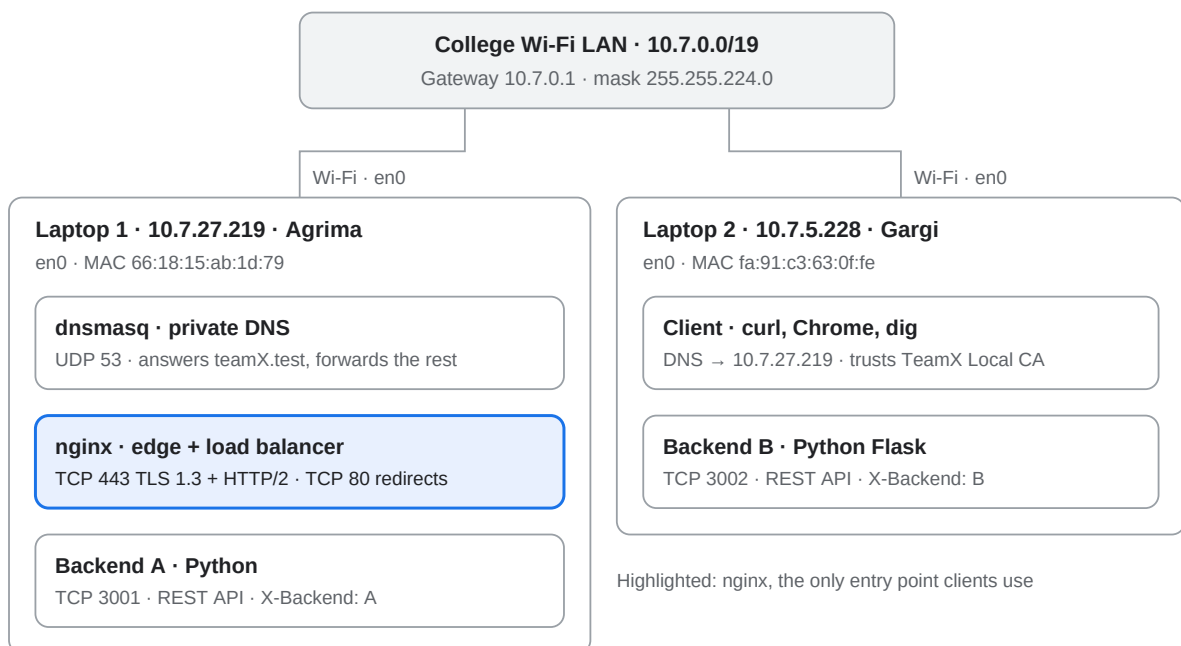


Figure 1. Network topology: 2 laptops, 1 Wi-Fi LAN, 5 services

Both laptops join the same college Wi-Fi, so they can talk to each other directly. A ping between them comes back with TTL 64, which means no router sits in between. Clients only ever talk to nginx on Laptop 1; they never contact the backends directly.

2. Who does what

Laptop 1 does the heavy lifting; Laptop 2 plays the user and hosts the second backend.

The two laptops

	Laptop 1	Laptop 2
Person	Agrima	Gargi
IP address	10.7.27.219	10.7.5.228
Subnet mask	255.255.224.0 (/19)	255.255.224.0 (/19)
Gateway (router)	10.7.0.1	10.7.0.1
Network interface	en0 (Wi-Fi)	en0 (Wi-Fi)
MAC address on the network	66:18:15:ab:1d:79	fa:91:c3:63:0f:fe
Roles	DNS server, edge (nginx), Backend A	Backend B, test client

What runs where

Service	Runs on	Port	In plain words	Real-world equivalent
dnsmasq (DNS)	Laptop 1	53 / UDP	The phonebook: turns <code>app.teamX.test</code> into <code>10.7.27.219</code> . Other names (like <code>google.com</code>) it passes on to Google's DNS.	AWS Route 53
nginx (edge)	Laptop 1	443 / TCP (80 redirects to 443)	The front door: handles HTTPS, then hands each request to Backend A or B in turn.	AWS Load Balancer
Backend A	Laptop 1	3001 / TCP	A small Python API. Labels its replies X-Backend: A.	An app server
Backend B	Laptop 2	3002 / TCP	A small Flask API, written separately. Labels its replies X-Backend: B.	An app server
Client	Laptop 2	Random (ephemeral)	<code>curl</code> , Chrome and <code>dig</code> , set to use Laptop 1 for DNS and to trust our certificate authority.	The end user

Our domain: `app.teamX.test` (and `api.teamX.test`), both pointing at nginx on Laptop 1. We used `.test` because it is reserved for testing and can never clash with a real website; `.local` is avoided because macOS uses it for local device discovery (mDNS).

The college Wi-Fi hands out addresses automatically, so we check both IPs at the start of every session.

3. Request flow

One request crosses DNS, TCP, TLS and HTTP before reaching a backend

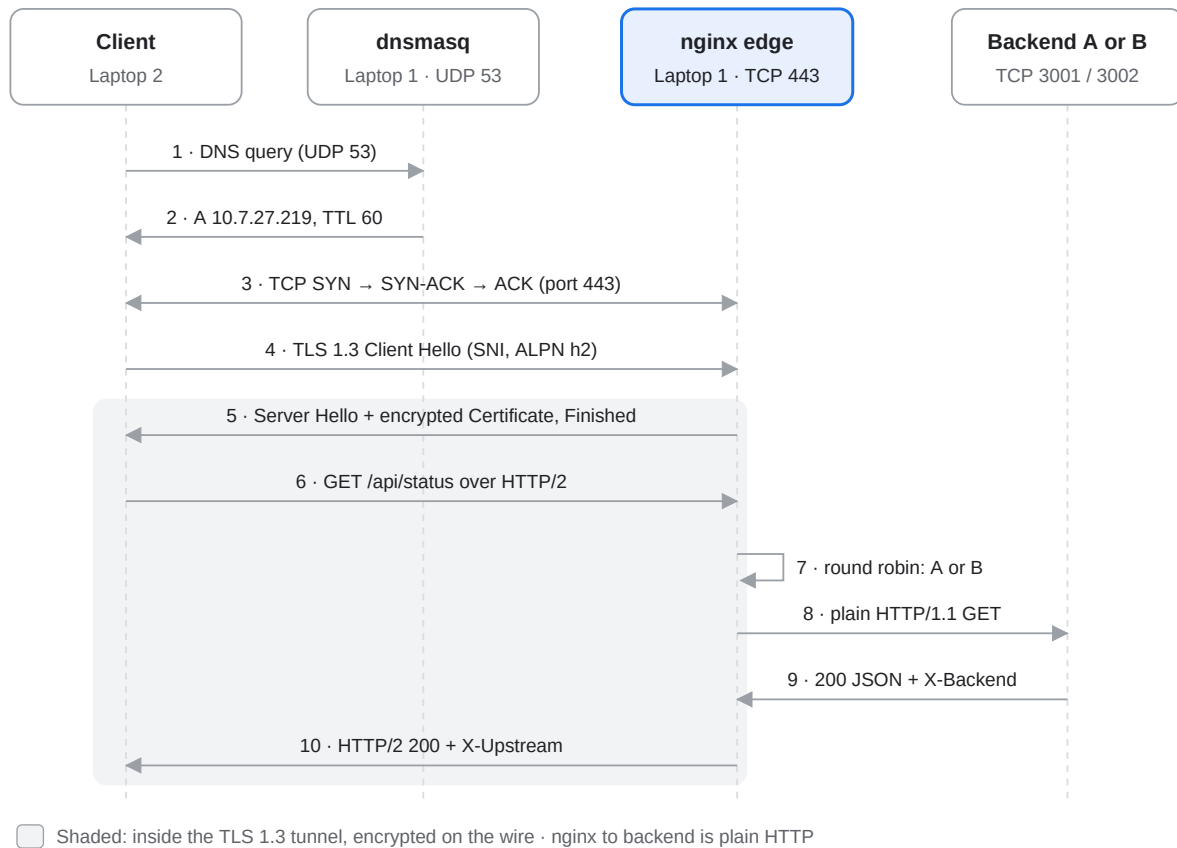


Figure 2. Request flow: 10 steps from name lookup to response

Here is what happens, step by step, when Laptop 2 runs `curl`

`https://app.teamX.test/api/status`:

- 1 “Where is app.teamX.test?”** Laptop 2 sends a DNS question to Laptop 1 on UDP port 53.
- 2 “It’s at 10.7.27.219.”** dnsmasq answers from its own records and says the answer is good for 60 seconds.
- 3 Knock on the door.** Laptop 2 opens a TCP connection to nginx on port 443 with the three-way handshake: SYN, SYN-ACK, ACK.
- 4 “Let’s talk securely.”** The TLS 1.3 Client Hello names the site it wants (SNI) and offers HTTP/2. This part is still readable on the network.
- 5 “Here’s my ID.”** nginx replies with its Server Hello and its certificate. From this point everything is encrypted, so even the certificate is hidden in TLS 1.3. Laptop 2 checks the certificate was signed by our TeamX Local CA, which it trusts.
- 6 The actual question.** Laptop 2 sends `GET /api/status` over HTTP/2, inside the encrypted tunnel.
- 7 Pick a backend.** nginx takes turns (round robin): this time A, next time B.

- 8 Pass it on.** nginx opens a second, separate TCP connection to the chosen backend and forwards the request as plain HTTP/1.1. It adds the client's real IP in **X-Forwarded-For**.
- 9 The answer.** The backend replies with JSON and its label, **X-Backend: A or B**.
- 10 Back to the user.** nginx encrypts the reply and sends it back over HTTP/2, adding **X-Upstream** to show which backend it used.

So the request is encrypted between the user and nginx, and plain between nginx and the backends. That is what “TLS terminates at the edge” means.

4. Which layer does what

Each protocol in our request has one job. Each layer carries the one above it.

Layer (OSI)	Layer (TCP/IP)	Protocol	Its job in our project	Port	Where we proved it
7 Application	Application	DNS	Turns app.teamX.test into 10.7.27.219	53	Wireshark DNS capture (G1)
7 Application	Application	HTTP/2 and HTTP/1.1	The request and the JSON reply; HTTP/2 to nginx, HTTP/1.1 from nginx to the backends	443, 3001, 3002	curl -v (E5), backend stream (G5)
5–6 Session / Presentation	Between Application and Transport	TLS 1.3	Encrypts the connection and proves nginx is really app.teamX.test	443	TLS handshake captures (G3a–G3c)
4 Transport	Transport	TCP	Reliable, ordered delivery for HTTPS and the backend connections	443, 3001, 3002	Three-way handshake (G2)
4 Transport	Transport	UDP	Quick one-question, one-answer delivery for DNS	53	DNS capture (G1)
3 Network	Internet	IP and ICMP	Addresses (10.7.x.x) and ping	none	Ping (A2)
2 Data link / 1 Physical	Link	Ethernet frames over Wi-Fi	Delivers frames between the two MAC addresses	none	Ethernet header in every capture

Ports in one sentence: the IP finds the laptop, the port finds the program. The server side always uses a fixed “well-known” port (53, 443, 3001, 3002); the client side uses a random temporary port the operating system picks. For example, one HTTPS connection in our capture went from **10.7.5.228:54556** to **10.7.27.219:443**, and nginx’s connection to Backend B went from **10.7.27.219:51560** to **10.7.5.228:3002**.

HTTP/3 is explanation-only for this project: it replaces TCP + TLS with a single encrypted protocol over UDP (QUIC).

5. Why we built it this way

One front door. Clients only know the name `app.teamX.test`, which points at nginx. Behind it we can add, remove or move backends without the user noticing.

Our own certificate authority. We created a “TeamX Local CA” with OpenSSL, used it to sign a certificate for `app.teamX.test` and `api.teamX.test`, and told both laptops to trust it. That is the same chain of trust the real internet uses, so browsers and curl show no warnings and we never need the `-k` shortcut. Private keys stay on Laptop 1 and are never shared or committed.

HTTPS ends at nginx. nginx decrypts the request and talks plain HTTP to the backends over our private LAN, so the backends need no certificates. In Phase 2, a firewall will make sure only nginx can reach the backend ports.

Taking turns, with a safety net. nginx sends requests to A and B in turn (round robin). If a backend refuses a connection, nginx retries the request on the other one and skips the broken backend for 10 seconds (`max_fails=1 fail_timeout=10s`). If both are down, nginx says so clearly with 502 Bad Gateway.

Caching the right things. The home page (`/`) can be cached for 60 seconds and carries the same version tag (ETag `"home-v1"`) on both backends, so a browser that already has it gets a quick “304 Not Modified” from either one. The status page (`/api/status`) is never cached because it is live data.

Two separate backends. Backend A is plain Python; Backend B is Flask, written by Gargi. They agree only on the endpoints, the `X-Backend` header and the ETag, which shows the edge does not care how a backend is built.

6. When something breaks

We broke the system on purpose to check each layer fails on its own. Screenshots are in the Evidence folder (H1–H5).

What we broke	What we saw	Layer that failed
Laptop 2 pointed at Google's DNS instead of ours	Name not found (NXDOMAIN), but ping and port 443 still worked	DNS
DNS record changed to the wrong IP	Name resolved, then “connection refused” at the wrong machine	DNS data
Backend A stopped	Every request still worked, served by B	One app server; nginx covered it
Both backends stopped	DNS, TCP and TLS fine, then 502 Bad Gateway from nginx	Application, behind the edge
Wrong port (8443)	Laptop reachable, port 443 fine, 8443 refused	Transport (no program on that port)

7. Known limits

- **Laptop 1 is a single point of failure.** It runs both the only DNS server and the only nginx. Phase 2 adds a backup DNS server and a standby nginx on Laptop 2.
- **Addresses can change.** College Wi-Fi hands out IPs automatically. If one changes, we update the DNS records, the nginx backend list and Laptop 2's DNS setting. The certificate does not need remaking, because it is tied to names, not IPs.